

Low-Poly Image Generation

Algorithm and Implementation Case Study

Norbert Fila

Technical project documentation

Keywords

low-poly rendering, Delaunay triangulation, image processing, C, C++, Slint

[Source code](#) [Author website](#) [Contact](#)

Contents

1. Abstract	3
2. Introduction	3
2.1. Problem Definition	3
2.2. Basic Terms	3
2.3. Scope and Design Goals	3
2.4. Paper Structure	3
3. System Overview and Data Pipeline	3
3.1. Component Structure	3
3.2. Processing Pipeline	3
3.3. GUI Execution	4
3.4. Technology Context	5
4. Mathematical and Geometric Foundations	5
4.1. Image Coordinates	5
4.2. Orientation Test	5
4.3. Triangle Centroid	6
5. Point Generation	7
6. Incremental Delaunay Triangulation	8
6.1. What Is a Triangulation?	8
6.2. What Is a Delaunay Triangulation?	8
6.3. Super-Triangle Initialization	8
6.4. Point Insertion and Cavity Construction	8
6.5. Retriangulation and Cleanup	8
6.6. Complexity of the Implemented Triangulation	8
7. Triangle Color Sampling and Rasterization	10
7.1. Rasterization Cost	11
8. Implementation and Software Integration	12
8.1. C Processing Core	12
8.2. C++ Service Layer	12
8.3. Slint GUI	12
8.4. Build Structure	12
9. Results and Observations	13
9.1. Point Count Comparison	13
9.2. Seed Reproducibility	15
9.3. Runtime Benchmark	15
10. Limitations and Possible Improvements	16
11. Conclusion	17
Bibliography	17

1. Abstract

This paper describes the algorithm and implementation of a low-poly image generator. It explains point generation, Delaunay triangulation, triangle rendering, and the data flow through the application. The document focuses on how the implementation works rather than presenting a course assignment or a formal academic evaluation.

2. Introduction

2.1. Problem Definition

The application transforms a raster image into an approximation composed of filled triangles. The output should preserve the main visual structure of the input while using fewer geometric elements.

2.2. Basic Terms

An image is a raster when it is stored as a rectangular grid of discrete pixels. Each pixel stores a color value at an integer coordinate. A triangle is a geometric primitive defined by three points, and a mesh is a collection of vertices, edges, and triangles that covers a region.

The low-poly effect is produced by replacing many individual pixels with larger colored triangle regions. The algorithm therefore moves between two representations: pixels in the input image and a geometric triangle mesh in the intermediate stage.

2.3. Scope and Design Goals

The case study covers the point-generation strategy, incremental Delaunay triangulation, centroid-based color sampling, rasterization, and the software pipeline that connects these stages. Reproducibility is controlled by an explicit random seed, while the point count controls the visual level of detail.

2.4. Paper Structure

The next section introduces the application pipeline. The following sections explain the geometry, triangulation, rendering, and integration layers in that order. Results then show how point count and seed affect the output.

3. System Overview and Data Pipeline

3.1. Component Structure

The implementation is organized into a C processing core, a C++ service layer, and two application interfaces: the command-line program and the optional Slint GUI.

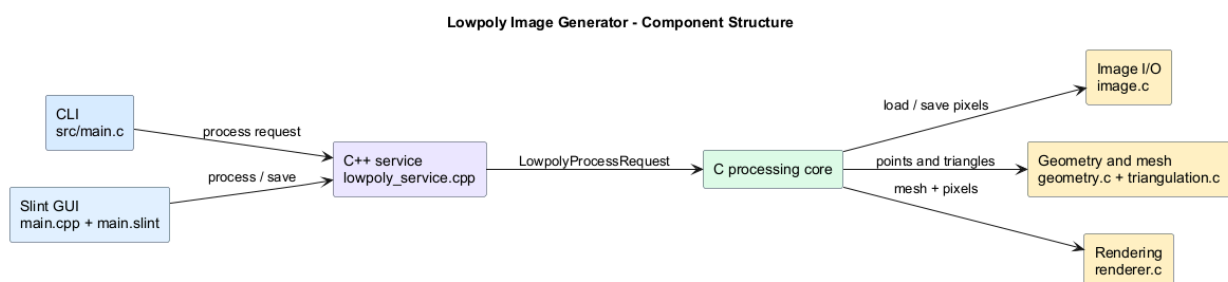


Figure 1: Application component structure and the boundary between the C core and the integration layers.

3.2. Processing Pipeline

The core pipeline is:

```
load image -> generate points -> triangulate -> sample colors -> rasterize
```

The `lowpoly_process()` function coordinates these stages. It validates the request first, loads the input into the arena, initializes the random generator, builds the mesh, and stores the rendered image and triangle count in the result. Failures are returned as explicit statuses instead of being mixed with image data. This gives both the CLI and the GUI one common processing path.

The central coordination is intentionally small:

```
1  LowpolyProcessStatus lowpoly_process(C
2      Arena *arena,
3      const LowpolyProcessRequest *request,
4      LowpolyProcessResult *result)
5  {
6      Image input = {0};
7      image_load_from_file(arena, request->input_path, &input);
8      Point *points = generate_random_points_with_corners(...);
9      Triangle *triangles = build_delaunay_triangulation(...);
10     result->output = render_low_poly_image(...);
11 }
```

Listing 1: The processing function connects the main algorithmic stages. Source: `src/processing.c`.

Lowpoly Image Generator - Processing Pipeline

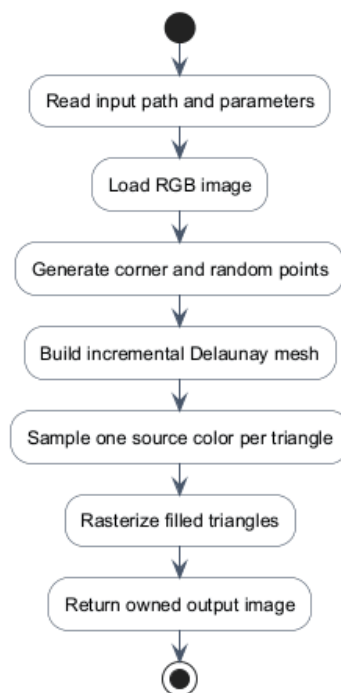


Figure 2: Main image-processing pipeline from input parameters to the owned output image.

3.3. GUI Execution

The GUI starts processing on a worker thread and sends the completed response back to the Slint event loop. This keeps image processing and file saving from blocking the interface.

The selected path, point count, and seed mode are collected by Slint. The C++ layer validates the values and creates a `ProcessRequest`. Once processing finishes, the response is kept alive for preview and can later be written as PNG or JPEG.

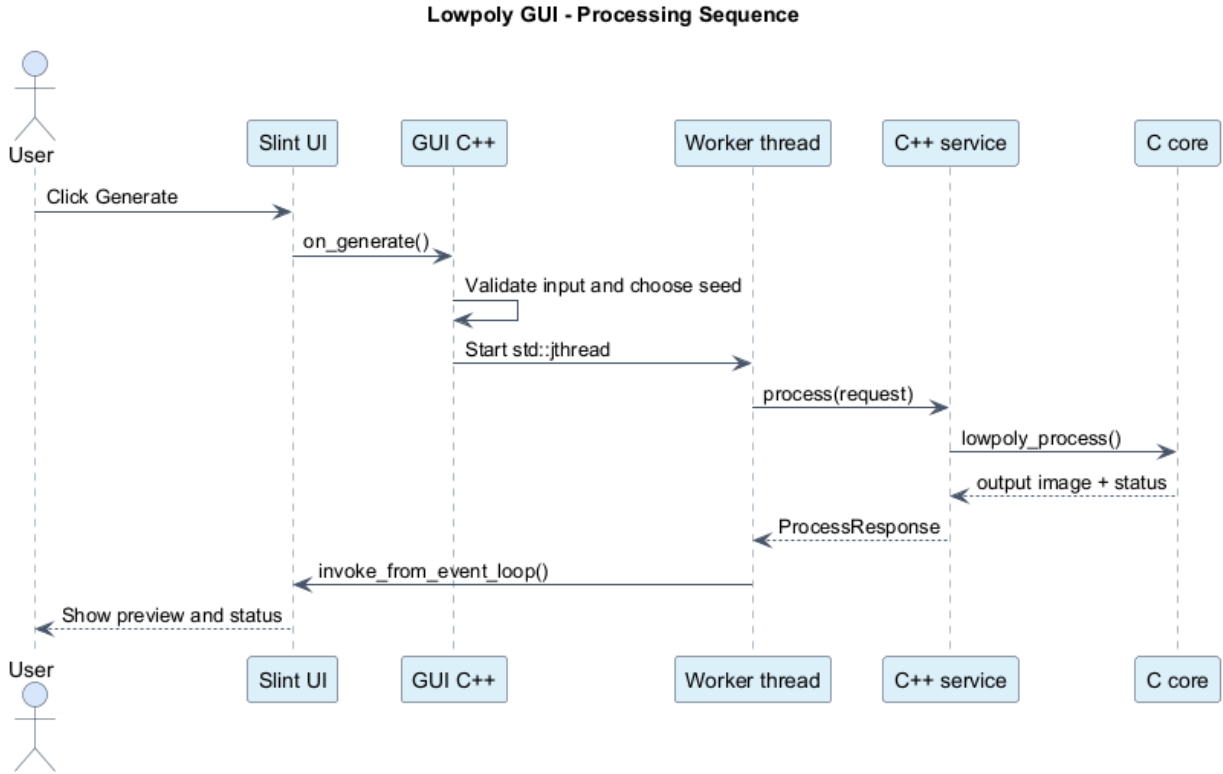


Figure 3: GUI processing sequence with background work and the Slint event loop.

3.4. Technology Context

Slint is a declarative framework for building native user interfaces. The GUI describes visual state and callbacks in `.slint`, while C++ connects those callbacks to the processing service. It was selected because it provides a small cross-platform desktop UI without moving the image algorithm out of the C/C++ application. The project uses the documented C++ integration model provided by Slint [1].

CMake controls the target graph and dependency configuration [2]. The native file picker is provided by `nativefiledialog-extended` [3], and the compact image I/O layer uses `stb` libraries [4]. These tools are used by the implementation but are not the subject of this paper.

4. Mathematical and Geometric Foundations

4.1. Image Coordinates

The image is represented in pixel coordinates (x, y) . The valid range is bounded by the image width and height. The four image corners are always part of the generated point set so the mesh covers the complete image rectangle. For an image of width w and height h , the corners are $(0, 0)$, $(w - 1, 0)$, $(w - 1, h - 1)$, and $(0, h - 1)$.

4.2. Orientation Test

For points a , b , and c , the signed orientation is

$$\text{orient}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Its sign distinguishes clockwise and counterclockwise order and supports the triangle containment checks used during rasterization.

The same geometric convention is used when triangles are created during triangulation. Keeping a counterclockwise ordering makes the orientation and circumcircle predicates consistent across the pipeline.

4.3. Triangle Centroid

The renderer samples a triangle at its centroid:

$$g = \frac{a + b + c}{3}.$$

The resulting pixel color is used as the fill color for the complete triangle.

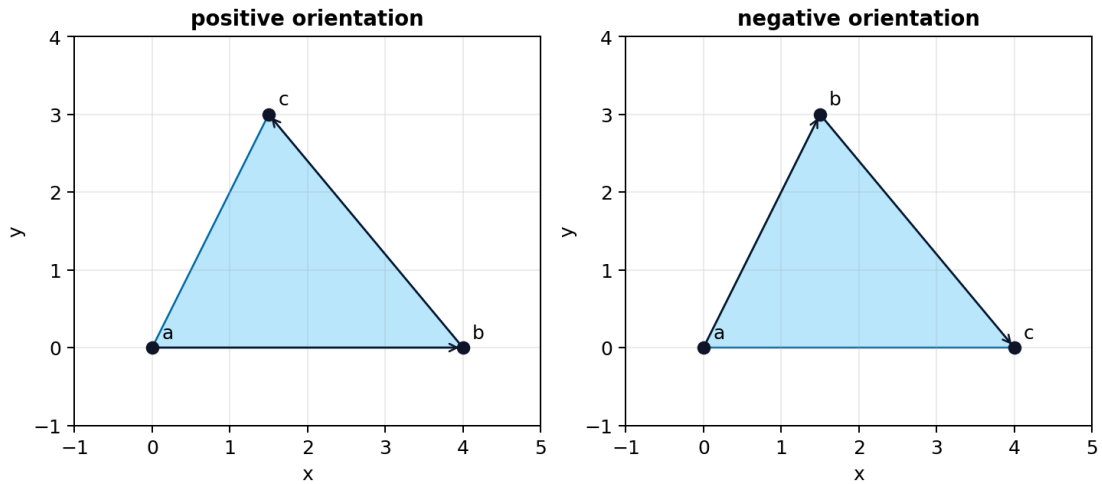


Figure 4: The signed orientation changes when the vertex order is reversed.

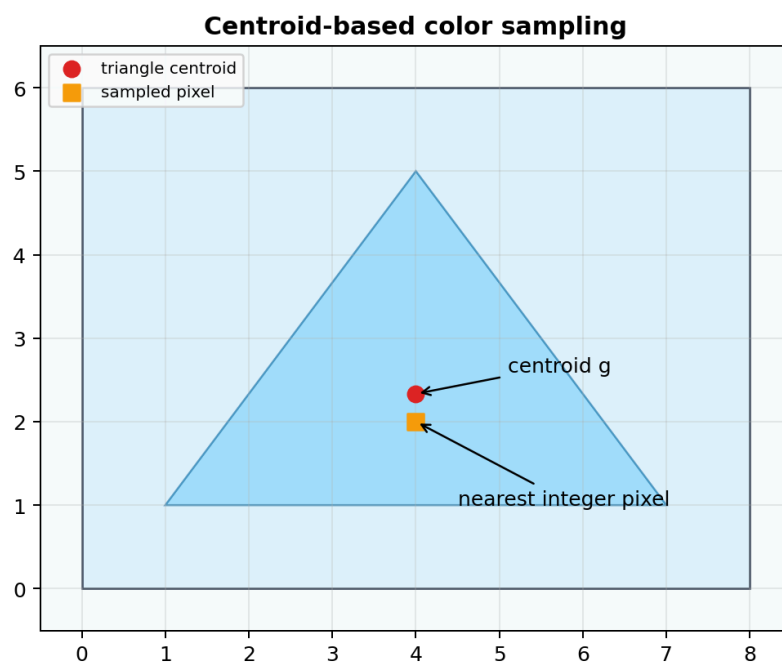


Figure 5: The triangle centroid is mapped to the integer pixel used for color sampling.

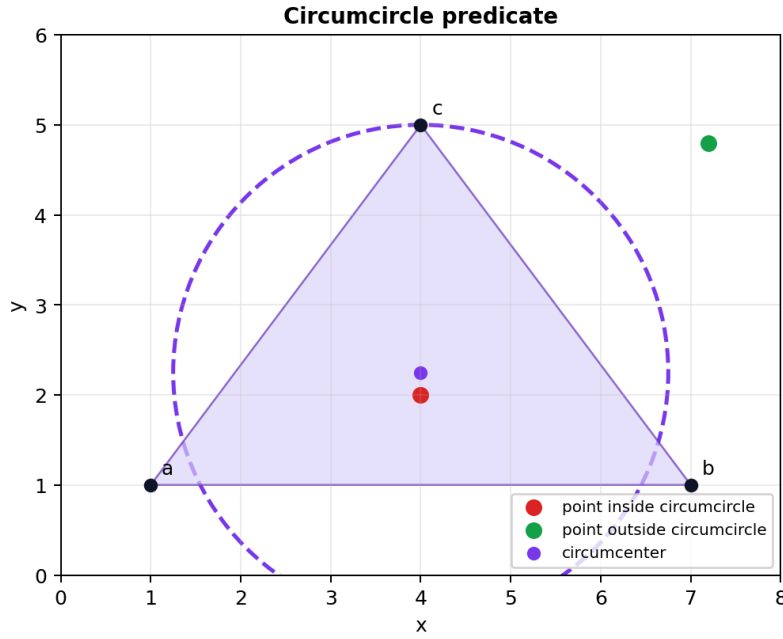


Figure 6: The circumcircle predicate separates an inside point from an outside point.

5. Point Generation

The requested point set consists of the four image corners and approximately uniformly distributed pseudo-random points inside the image bounds. The seed controls the random sequence, so the same image dimensions, point count, and seed produce the same point set.

The implementation uses a small xorshift32 generator, a shift-and-XOR PRNG family described and experimentally examined by Kent [5]. Each generated coordinate is reduced modulo the corresponding image dimension, while the corner points are inserted explicitly before the random points. This is simple, fast, and sufficient for distributing visual samples over the image area; it is not intended to be a cryptographic random source.

Point generation with xorshift32 seed 123

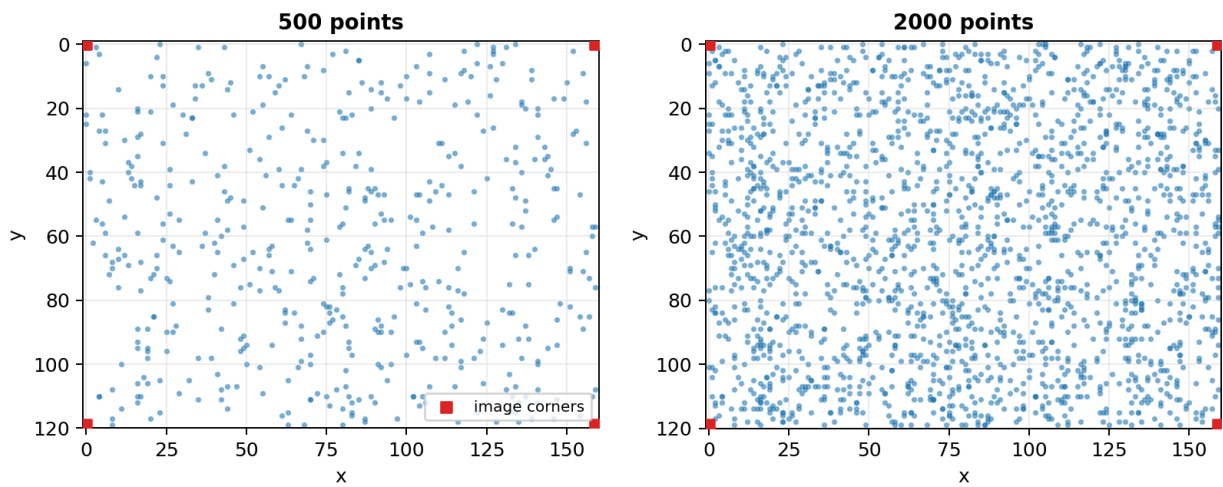


Figure 7: Generated point sets for two point counts using the same seed. Red markers are forced image corners.

```

1 points[0] = (Point){0, 0};
2 points[1] = (Point){width - 1, 0};
3 points[2] = (Point){width - 1, height - 1};
4 points[3] = (Point){0, height - 1};
5
6 for (int i = 4; i < requested_count; i++) {
7     points[i].x = random_range(random, width);
8     points[i].y = random_range(random, height);
9 }

```

Listing 2: The four corners are fixed before random interior points are generated. Source: src/triangulation.c.

6. Incremental Delaunay Triangulation

6.1. What Is a Triangulation?

A triangulation divides a set of points, or a bounded region containing those points, into non-overlapping triangles. The triangles share edges, and their union represents the covered region. Different triangulations can be built from the same points, including meshes with very narrow or uneven triangles.

6.2. What Is a Delaunay Triangulation?

A Delaunay triangulation is characterized by the empty-circumcircle property: the circumcircle of each triangle contains no other input point in its interior. This property tends to avoid unnecessarily narrow triangles and makes the mesh useful for interpolation and geometric rendering. The incremental construction used here follows the cavity-rebuilding approach associated with Bowyer and Watson [6], [7].

The term **Delaunay** is therefore introduced before the implementation details: the project is not merely connecting points arbitrarily; it is repeatedly preserving a geometric condition while the mesh grows.

6.3. Super-Triangle Initialization

The implementation first creates a large temporary triangle around the real points. Its vertices provide a simple initial mesh for point insertion. The temporary vertices are appended after the real points in the same point array and are identified by indices outside the real point range.

6.4. Point Insertion and Cavity Construction

For every real point, triangles whose circumcircles contain the point are removed. Their edges are collected into a polygon boundary. Edges shared by two removed triangles cancel, leaving only the boundary of the affected cavity. The edge cancellation is implemented by treating an edge and its reversed form as equivalent. A boundary edge therefore remains once, while an internal edge is removed when encountered for the second time.

6.5. Retriangulation and Cleanup

Each remaining boundary edge is connected to the inserted point. After all real points are processed, triangles that reference a super-triangle vertex are removed. Temporary cavity storage is allocated after an arena snapshot and released by rewinding the arena at the end of each insertion. This savepoint-style use of an arena follows the linear allocation and reset model described by Ginger Bill [8]. The triangle array itself is kept for the final mesh and is checked against a fixed capacity.

6.6. Complexity of the Implemented Triangulation

The implementation does not maintain triangle adjacency, a spatial index, or a priority structure. For every point p , the loop in `build_delaunay_triangulation` scans the current triangle array and evaluates the circumcircle predicate for each entry. When a triangle is removed, the last array entry is moved into its place, so removed entries are not scanned again during that same insertion. If T_p is the number of current triangles before inserting point p , the circumcircle-search cost is therefore

$$O\left(\sum_{p=1}^n T_p\right).$$

The triangle array is allocated with capacity $10 * n + 100$, where n is the requested real-point count. This is an implementation guard and memory budget, not an optimization of the search: the scan still visits the active array entries. Since a planar triangulation has a linear number of triangles for a non-degenerate point set, the scan is typically quadratic in n when the intermediate triangle count remains proportional to n , rather than linear or near-linear in the number of points.

There is a second cost hidden in cavity construction. `add_or_remove_edge()` searches the current polygon edge list linearly using an undirected edge comparison. If an insertion processes k bad triangles, it performs up to $3k$ such edge operations, and the repeated searches can cost $O(k^2)$ in the worst case. Consequently, the implementation has a practical typical cost dominated by the repeated triangle scans, while its worst-case bound can reach $O(n^3)$ when large cavities also make the edge-list searches quadratic. No claim of a faster asymptotic bound is made here: the bound follows from the actual flat arrays and linear searches in `src/triangulation.c`.

This design trades asymptotic performance for a small implementation. The swap-with-last deletion keeps the triangle array compact and avoids shifting elements, and the arena snapshot makes temporary cavity storage cheap to discard. The consequences are that the method is easy to inspect and has predictable fixed-capacity checks, but it can spend substantial time rescanning unaffected triangles as the point count grows.

Incremental Delaunay Triangulation - High-Level Flow

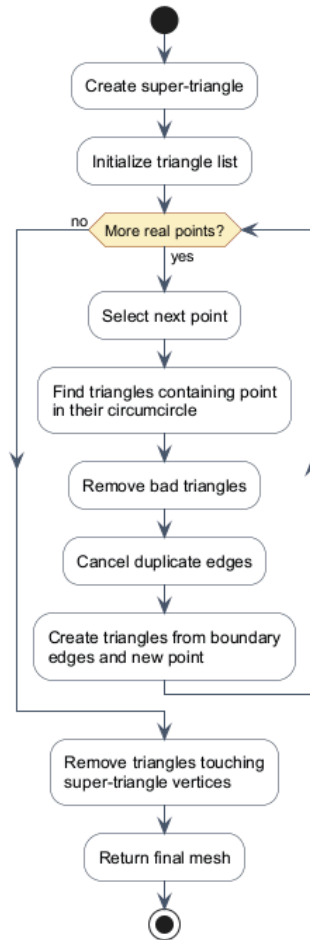


Figure 8: High-level flow of incremental Delaunay triangulation.

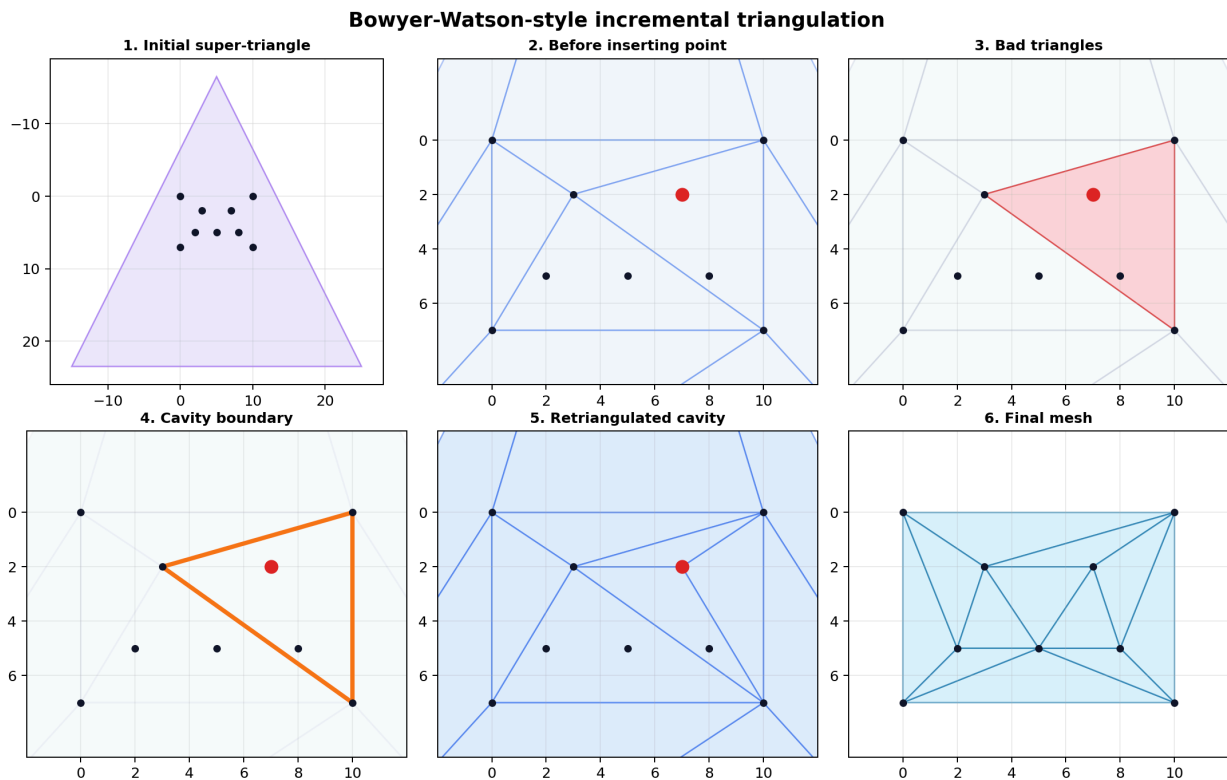


Figure 9: Six-stage walkthrough of one point insertion and the final super-triangle cleanup.

```

1 for (int i = 0; i < triangle_count;) {
2     if (is_point_inside_circumcircle(...)) {
3         add_or_remove_edge(polygon, &polygon_count, edge);
4         triangles[i] = triangles[triangle_count - 1];
5         triangle_count--;
6     } else {
7         i++;
8     }
9 }

```

Listing 3: Bad triangles are removed while their boundary edges are collected. Source: `src/triangulation.c`.

7. Triangle Color Sampling and Rasterization

For each final triangle, the renderer calculates a bounding box and tests pixel centers inside that box. Pixels contained by the triangle receive the color sampled at its centroid. This produces a raster image without requiring a separate texture or interpolation step. The bounding box is clipped to the image dimensions before scanning. Sampling pixel centers rather than pixel corners avoids treating a pixel as covered only because its top-left coordinate touches an edge.

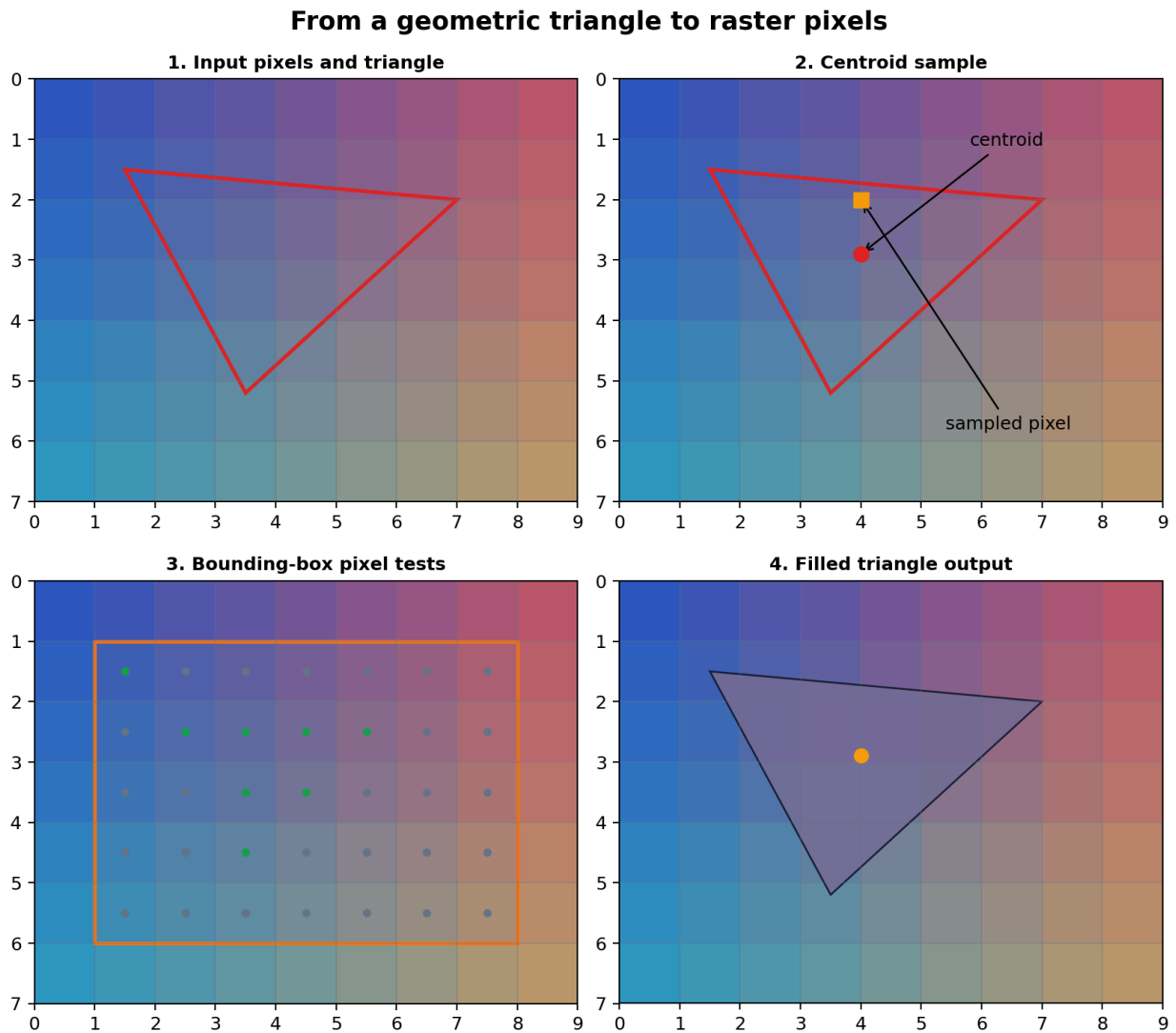


Figure 10: From a triangle over an input raster to bounding-box tests and filled output pixels.

```

1 for (int i = 0; i < triangle_count; i++) {
2     Triangle triangle = triangles[i];
3     Pixel color = sample_color_at_triangle_centroid(input, ...);
4     rasterize_filled_triangle(&output, a, b, c, color);
5 }

```

Listing 4: Each triangle receives a sampled color before it is rasterized. Source: src/renderer.c.

7.1. Rasterization Cost

The renderer uses a deliberately direct rasterization strategy. For every final triangle it computes a clipped axis-aligned bounding box, visits every pixel coordinate in that rectangle, and runs `geometry_is_point_inside_triangle()`. That predicate performs three orientation calculations and constant-time sign checks. If B_i is the number of candidate pixels in the clipped bounding box of triangle i , the rasterization work is

$$O\left(\sum_{i=1}^m B_i\right),$$

where m is the final triangle count. The renderer also allocates and clears the complete output image, which adds $O(WH)$ work for an image of width W and height H . The total rendering cost is therefore

$$O\left(WH + \sum_{i=1}^m B_i\right),$$

not simply $O(WH)$. The bounding boxes are often much smaller than the full image, but they may overlap heavily, so the same pixel coordinate can be tested for multiple triangles. The relevant quantity is the sum of bounding-box areas, not only the final number of covered pixels.

The centroid color lookup itself is constant time per triangle, because the centroid is converted to one clamped input-pixel coordinate. The simple bounding-box method therefore makes the implementation easy to follow and avoids a full-image scan for every triangle, but it does not use incremental edge functions, scanline coherence, or a tile/binning structure. Those omitted structures could reduce redundant tests for large images, at the cost of more state and more complicated boundary handling. The current implementation keeps the simpler trade-off explicit in `src/renderer.c`.

8. Implementation and Software Integration

8.1. C Processing Core

The C modules implement image handling, random generation, geometry, triangulation, rendering, and pipeline coordination. Temporary allocations use an arena, while processing returns status values for invalid requests and input or triangulation failures. The core does not depend on Slint or the desktop file dialog, so the same implementation is usable from the command-line target and from tests.

8.2. C++ Service Layer

The C++ service adapts the C pipeline to an owned request/response interface. It keeps the final image available for preview and later PNG or JPEG saving. This boundary also prevents the GUI from managing the lifetime of temporary C arena allocations directly.

8.3. Slint GUI

Slint owns the visual state and callbacks. The C++ layer handles native file dialogs, input validation, background work, image previews, and event-loop updates. The GUI is therefore an integration layer rather than a second implementation of the algorithm.

The long-running operation is kept outside the UI callback:

```
1 worker = std::jthread([weak_window, request] {
2     lowpoly::ProcessResponse response = service.process(request);
3     slint::invoke_from_event_loop([weak_window, response = std::move(response)] {
4         // Update the preview on the Slint event loop.
5     });
6 });
```

Listing 5: The GUI worker returns results to the Slint event loop. Source: `src/gui/main.cpp`.

8.4. Build Structure

CMake builds the C17 core, C++20 service, CLI, tests, and optional GUI. Slint and the native file dialog are fetched only for the GUI build. The explanatory diagrams use PlantUML [9], while the generated plots use Python and matplotlib [10].

Component	Responsibility	Language
Processing core	Image loading, geometry, triangulation, rendering	C
Service layer	Owned requests, responses, and image saving	C++
CLI	Argument parsing and batch processing	C
GUI	Controls, previews, dialogs, and background work	C++ / Slint

Component	Responsibility	Language
Build system	Targets, dependencies, and platform configuration	CMake

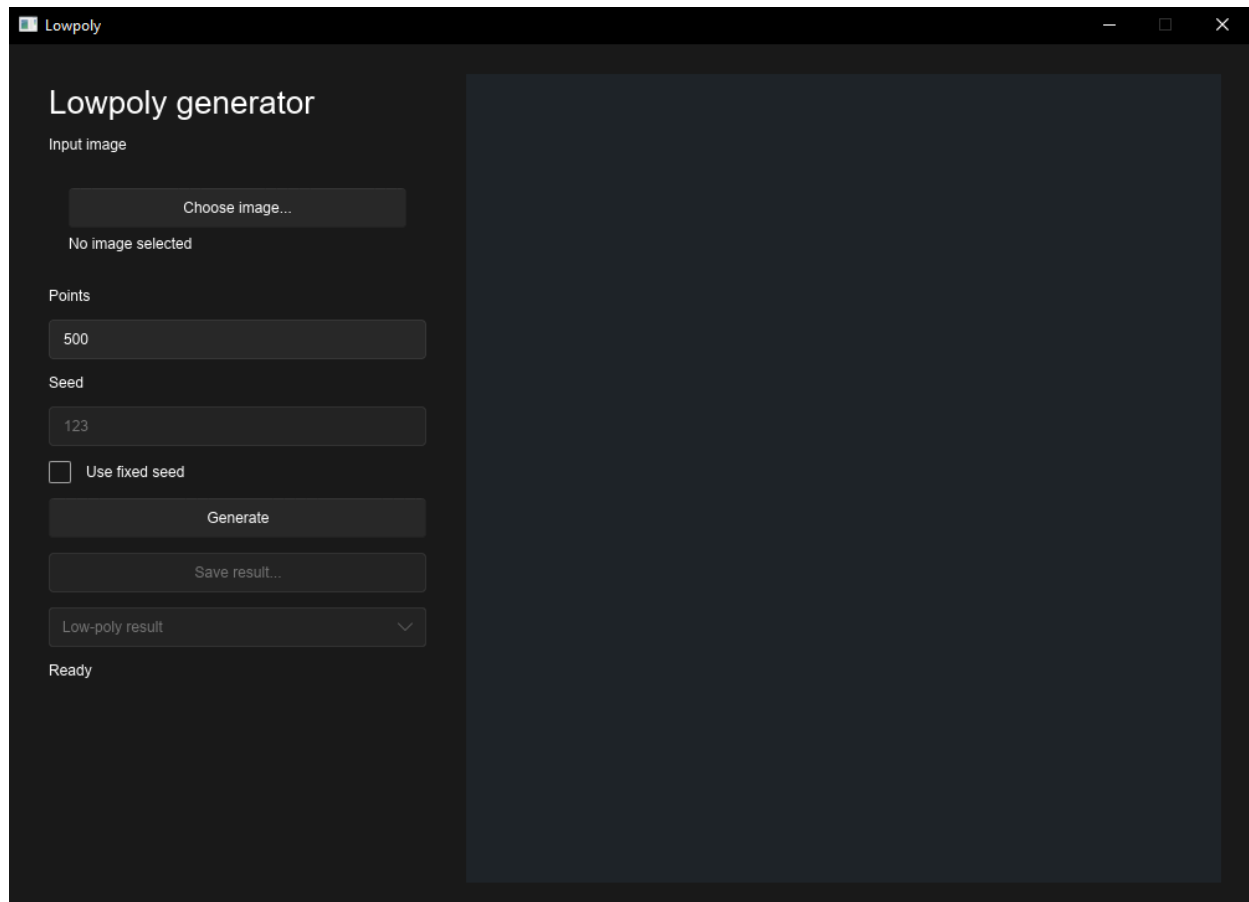


Figure 11: Initial state of the desktop GUI before an input image is selected. The preview area is empty and the controls are ready for a new request.

9. Results and Observations

9.1. Point Count Comparison

Increasing the point count creates smaller triangles and preserves more local detail. Lower point counts produce larger regions and a stronger geometric abstraction.

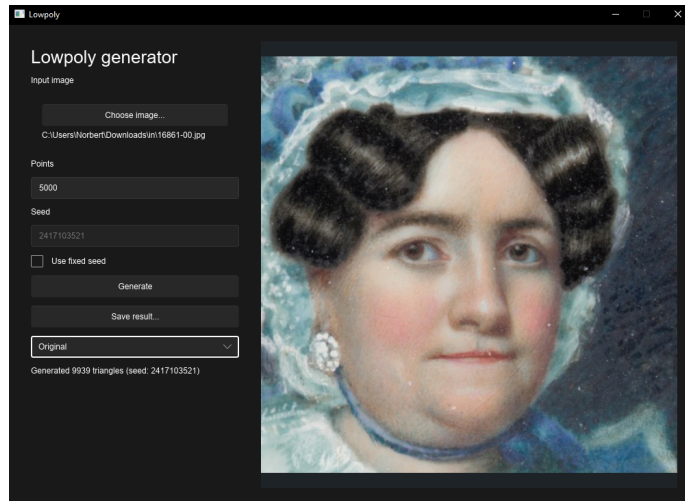


Figure 12: Example input image used for the output comparisons.

500 points	1000 points
2000 points	5000 points

Table 1: Effect of the requested point count on the output image.

The comparison shows the expected trade-off: increasing the point count reduces triangle size and preserves more local structure.

9.2. Seed Reproducibility

Changing the seed changes the interior points

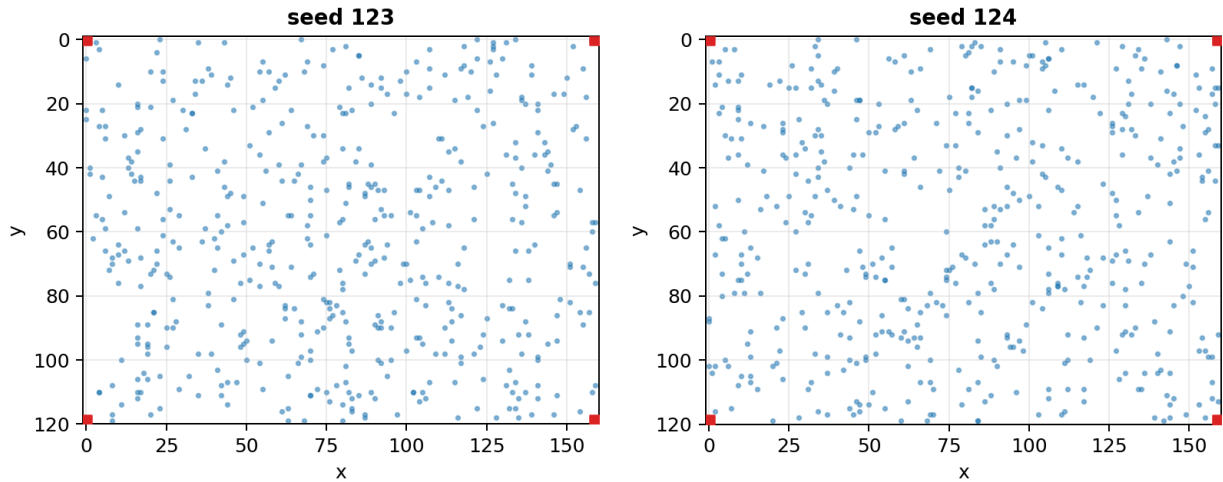


Figure 13: Changing the seed changes the interior points while preserving the image corners.



The two outputs generated with seed 123 are byte-for-byte identical. Changing the seed to 124 changes the interior point set and therefore changes the triangle arrangement and output image. The images were generated by `paper/scripts/generate_seed_comparison.py` from the CLI executable.

9.3. Runtime Benchmark

The complexity analysis predicts that the flat triangle scan and repeated edge searches should make larger point sets increasingly expensive. This prediction was tested using the real `lowpoly` CLI rather than a separate algorithm model. The benchmark used a Release build, the fixed input image `test_images/in/394547-00.jpg`, seed 123, two warmup runs, and ten measured runs for each point count. The tested values were 250, 500, 1000, 2000, 4000, and 8000 points. Each run performed the complete CLI pipeline: image loading, point generation, triangulation, rasterization, and PNG writing.

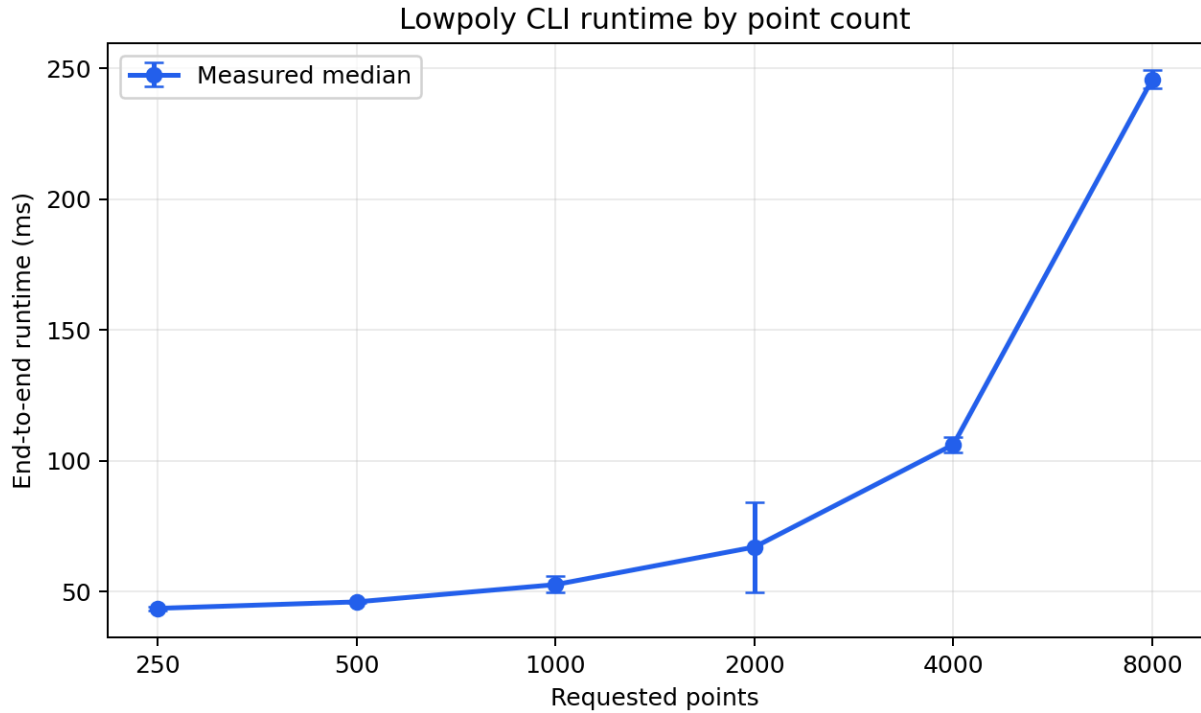


Figure 14: Median end-to-end CLI runtime by requested point count. Error bars show one standard deviation across 10 runs.

Points	Runs	Median (ms)	Std. dev. (ms)
250	10	43.609	0.807
500	10	46.081	0.499
1000	10	52.685	3.111
2000	10	67.051	17.141
4000	10	106.219	2.937
8000	10	245.861	3.364

The median runtime increases from 43.609 ms at 250 points to 245.861 ms at 8000 points. The increase is not perfectly quadratic across the full range because image allocation, image decoding, PNG writing, and rasterization add costs that do not follow the triangulation scan. The stronger increase at the upper point counts is nevertheless consistent with the implementation analysis: the triangulation work repeatedly scans a growing flat triangle array, while the edge list is searched linearly during cavity construction.

These measurements are end-to-end runtime measurements, not an isolated timer around `build_delaunay_triangulation()`. They therefore support the predicted scaling of the complete application pipeline while avoiding the false claim that every measured millisecond belongs to triangulation. The raw observations and summary were generated by `paper/scripts/benchmark_runtime.py` and `paper/scripts/plot_benchmark.py`.

10. Limitations and Possible Improvements

The current method uses one centroid pixel to represent each triangle and does not deliberately place points along image edges. It is CPU-based, and larger point counts increase memory use and processing time. These choices keep the implementation compact and understandable, but leave room for edge-aware point placement, area-average color sampling, and parallel rendering.

11. Conclusion

The application transforms an input raster into a low-poly approximation in a sequence of explicit stages. Image corners and seeded random points define the geometric samples. Incremental Delaunay triangulation connects those samples into a mesh, and centroid sampling assigns one source color to each triangle. Finally, bounded pixel tests rasterize the mesh into the output image.

The implementation keeps this algorithm in a reusable C core. A C++ service provides ownership and format boundaries, while the Slint GUI supplies an interactive interface and runs long operations away from the event loop. This separation makes the algorithm easier to test, explain, and reuse independently of the desktop interface.

Bibliography

- [1] Slint Developers, “Slint Documentation.” Accessed: Oct. 06, 2026. [Online]. Available: <https://docs.slint.dev/latest/docs/slint/>
- [2] Kitware, “CMake Documentation.” Accessed: Oct. 06, 2026. [Online]. Available: <https://cmake.org/cmake/help/latest/>
- [3] B. Tzy, “nativefiledialog-extended.” Accessed: Oct. 06, 2026. [Online]. Available: <https://github.com/btzy/nativefiledialog-extended>
- [4] S. Barrett, “stb Single-File Public Domain Libraries.” Accessed: Oct. 06, 2026. [Online]. Available: <https://github.com/nothings/stb>
- [5] A. Kent, “PRNGs & Randograms.” Accessed: Oct. 06, 2026. [Online]. Available: <https://alexkent3141.github.io/2025/07/01/randograms.html>
- [6] A. Bowyer, “Computing Dirichlet tessellations,” *The Computer Journal*, vol. 24, no. 2, pp. 162–166, 1981, doi: [10.1093/comjnl/24.2.162](https://doi.org/10.1093/comjnl/24.2.162).
- [7] D. F. Watson, “Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes,” *The Computer Journal*, vol. 24, no. 2, pp. 167–172, 1981, doi: [10.1093/comjnl/24.2.167](https://doi.org/10.1093/comjnl/24.2.167).
- [8] Ginger Bill, “Memory Allocation Strategies - Part 2: Linear/Arena Allocators.” Accessed: Oct. 06, 2026. [Online]. Available: <https://www.gingerbill.org/article/2019/02/08/memory-allocation-strategies-002/>
- [9] PlantUML Team, “PlantUML Language Reference Guide.” Accessed: Oct. 06, 2026. [Online]. Available: <https://plantuml.com/guide>
- [10] Matplotlib Development Team, “Matplotlib Documentation.” Accessed: Oct. 06, 2026. [Online]. Available: <https://matplotlib.org/stable/>